Penetration Testing Report

OWASP Juice Shop Security Assessment & Manual Findings

TARGET ENVIRONMENT

Juice-shop v20.2

ENGAGEMENT DATE

October 2026

LEAD ASSESSOR

Rohit Dalal

CLASSIFICATION

Confidential

1. EXECUTIVE SUMMARY

During October 2026, an external security assessment was performed against the **OWASP Juice Shop** web application. The objective of this engagement was to evaluate the application's overall resilience against standard vulnerability classes, simulate real-world attacks, and provide actionable remediation insights.

The assessment uncovered critical weaknesses allowing authentication bypass via SQL Injection, Stored Cross-Site Scripting (XSS), and

Insecure Direct Object References (IDOR). Immediate remediation is advised to secure customer records and maintain platform integrity.

2. SCOPE & METHODOLOGY

Testing followed strict methodologies mapped to the **OWASP Top 10** and **CWE** guidelines. Assessment activities included service enumeration, manual application analysis, and payload injection.

HOST & SERVICE DISCOVERY

TARGET URL / HOST	PORT / SERVICE	DISCOVERED COMPONENT	STATUS
`http://localhost:3000` OR `http://juiceshop.com:3000`	TCP / 3000	Node.js / Express (Juice Shop Instance)	Active / In Scope

3. FINDINGS REGISTER

The table below summarizes all vulnerabilities identified during the assessment, ordered by vulnerability class and severity rating.

ID	VULNERABILITY TITLE	SEVERITY	CWE MAPPING
SEC-01	SQL injection in authentication mechanism	CRITICAL	CWE-89
SEC-02	JWT auth bypass via flawed signature verification	CRITICAL	CWE-347
SEC-03	SSRF vulnerability in user profile update	HIGH	CWE-918

ID	VULNERABILITY TITLE	SEVERITY	CWE MAPPING
SEC-04	Reflexted Cross-Site Scripting (XSS) in Search Parameter	MEDIUM	CWE-79
SEC-05	Unauthorized user can access order information of other users	MEDIUM	CWE-639
SEC-06	Forged User-feedback / forged product review	MEDIUM	CWE-289
SEC-07	CSRF in password change	MEDIUM	CWE-352
SEC-08	Insecure Direct Object Reference (IDOR) leads to view another users' shopping baskets	MEDIUM	CWE-639
SEC-09	Unauthorized order tracking of other users	MEDIUM	CWE-639
SEC-10	FTP Dir listing is enabled and contains critical files (`.kdbx`)	LOW	CWE-538

4. DETAILED TECHNICAL FINDINGS

SEC-01: SQL Injection via Authentication Form

SEVERITY
Critical (9.8)

VULNERABILITY CLASS

SQL Injection

CWE IDENTIFIER

CWE-89

AFFECTED ENDPOINT

`/rest/user/login`

DESCRIPTION

The application backend incorporates user-supplied email input directly into database query statements without prior sanitization or parameterization, allowing user-controlled command manipulation.

STEPS TO REPRODUCE

1. Open the login page at `http://localhost:3000/#/login` .

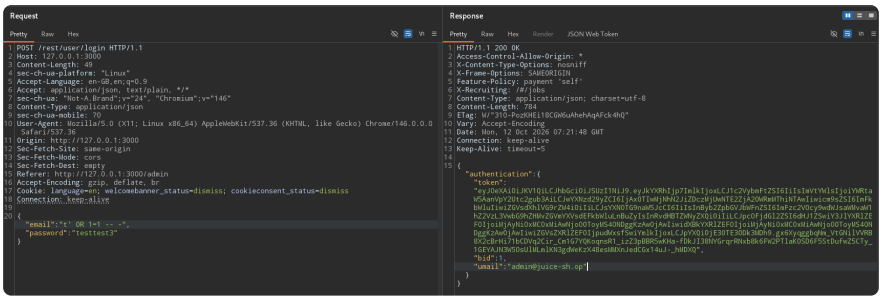
2. Intercept the request in burpsuite and replace the string payload into the email form field:

```
' OR 1=1 -- -
```

3. Submit arbitrary text into the password field and click log in.

4. The database authenticates the request as true, returning access tokens for the primary administrator profile.

PROOF



IMPACT

Full authentication bypass, yielding complete control over administrative dashboard capabilities and backend database.

REMEDIATION

SHORT/LONG TERM: Implement parameterized queries (prepared statements) for all database interactions.

SEC-02: JWT Authentication Bypass via Flawed Signature Verification

SEVERITY
Critical (9.8)

VULNERABILITY CLASS
JWT Signature Verification Bypass

CWE IDENTIFIER
CWE-347

AFFECTED ENDPOINT
`JWT-protected routes/endpoints`

DESCRIPTION

The application accepts JWTs without correctly validating the token signature. Depending on the configured challenge, this may allow use of an unsigned token with `alg: none`. An attacker can therefore forge token claims, including identity and administrative privileges.

STEPS TO REPRODUCE

1. Authenticate as a normal user and copy the JWT from the `Authorization: Bearer` header or authentication `token` from the cookie header.

2. Decode the JWT and modify the payload to impersonate another user or administrator, for example:

```
{
  "id": 1,
  "email": "admin@juice-sh.op",
  "username": "administrator"
}
```

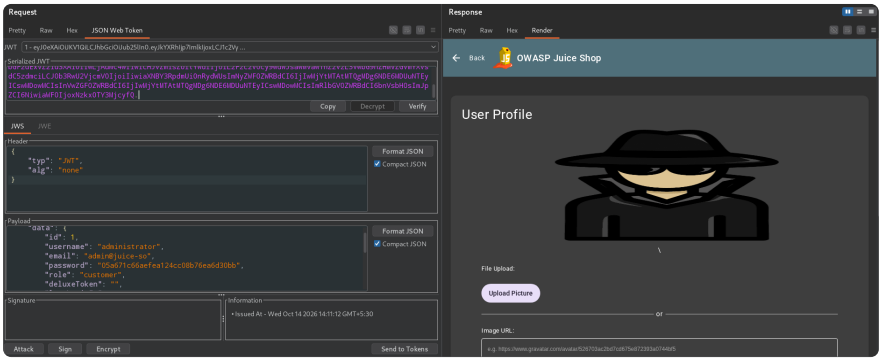
3. Set `alg: none` with an empty signature, or sign it using HS256 with the server's exposed RSA public key as the HMAC secret.

4. Send the forged token to a protected endpoint:

```
GET /profile
Authorization: Bearer <forged_jwt>
```

5. The server accepts the forged token and returns identity or authorization data for the impersonated user.

PROOF



IMPACT

Complete authentication bypass and potential privilege escalation to administrative access. An attacker can impersonate any user, access sensitive account data, and invoke authorized application functionality.

REMEDIATION

SHORT/LONG TERM: Enforce an allow-list of JWT algorithms and reject `algo: none`, mixed algorithm usage, and algorithm-confusion attempts. Validate signatures using a

properly protected key, verify issuer, audience, expiration, and required claims, and never derive authorization decisions from unverified token claims.

SEC-03: Server-Side Request Forgery via Profile Update

SEVERITY
High (8.6)

VULNERABILITY CLASS
Server-Side Request Forgery (SSRF)

CWE IDENTIFIER
CWE-918

AFFECTED ENDPOINT
`/profile/image/url` **(profile update functionality)**

DESCRIPTION

The profile update feature accepts a user-controlled image URL and retrieves it from the server without sufficient validation. This allows an attacker to force the backend to issue requests to internal network resources, cloud metadata endpoints, or arbitrary external hosts.

STEPS TO REPRODUCE

1. Log in and navigate to the user profile page.

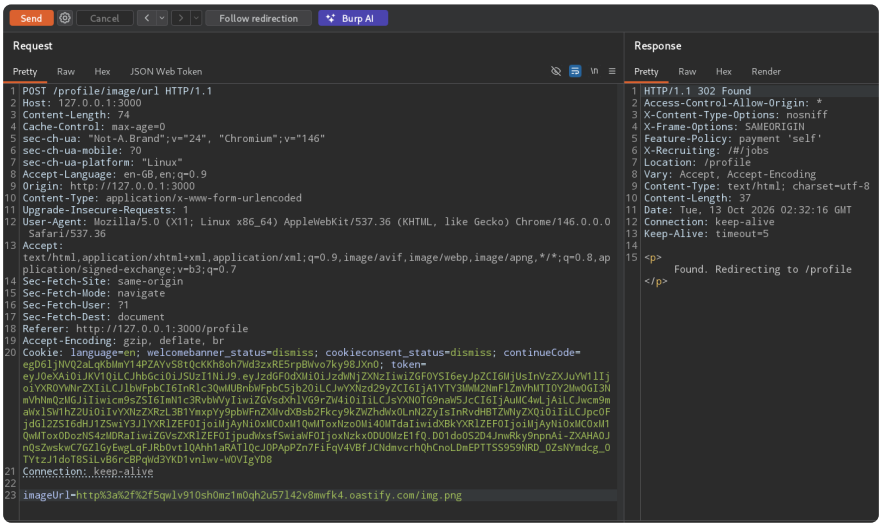
2. In the profile "image URL" field, supply a URL pointing to an attacker-controlled listener or internal service:

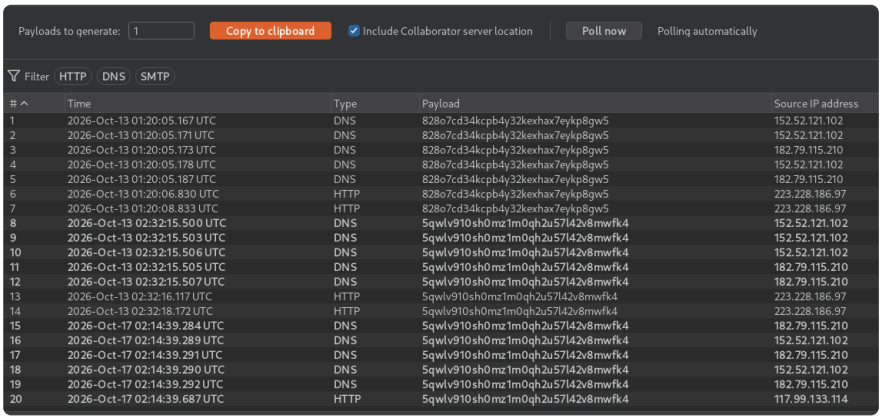
```
http://<REDACTED>.oastify.com/profile.png
```

3. Click on "Link Image" button.

4. Observe that the server makes a request to the supplied URL. Using an external listener such as Burp Collaborator or a local HTTP server confirms the server-side request.

PROOF





IMPACT

An attacker may scan internal services, access restricted internal endpoints, retrieve sensitive configuration data, or interact with cloud instance metadata services. This can lead to further compromise of the application or underlying infrastructure.

REMEDIATION

SHORT TERM: Verify user supplied input, use an allow-list of trusted hosts and validate the origin of the request.
LONG TERM: Disable URI schemas which are not required.

SEC-04: Reflected Cross-Site Scripting in Search Feature

SEVERITY

High (7.1)

VULNERABILITY CLASS

Reflected Cross-Site Scripting (XSS)

CWE IDENTIFIER

CWE-79

AFFECTED ENDPOINT

`/#/search?q=`

DESCRIPTION

User-supplied search input is reflected into the rendered search-results page without adequate HTML encoding, output escaping, or sanitization. As a result, attacker-controlled markup or JavaScript is executed in the browser context of a victim who opens a crafted search URL.

STEPS TO REPRODUCE

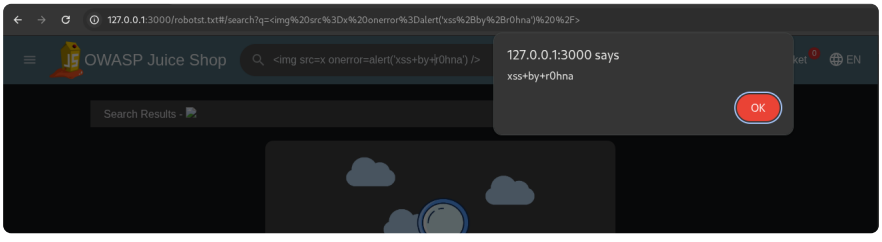
1. Open the Juice Shop search page.

2. Enter the following payload into the search field:

```
<img%20src%3Dx%20onerror%3D"alert(`xss+by+r0hna`)">
```

3. Submit the search. The payload is reflected in the results page and executed by the browser.

PROOF



IMPACT

An attacker can execute arbitrary JavaScript in a victim's browser, potentially stealing session tokens, performing actions on behalf of the user, modifying page content, or redirecting the victim to malicious sites.

REMEDIATION

LONG TERM: Use AngularJs contextual encoding for all user supplied input, use `ngSanitize` modules.
Implement a strict Content Security Policy, use backend `DOMPurify or sanitize-html` to clean up user input before processing.

SEC-05: Unauthorized Access to Other Users' Order Information

SEVERITY
High (8.2)

VULNERABILITY CLASS
Broken Access Control / IDOR

CWE IDENTIFIER
CWE-639

AFFECTED ENDPOINT
`/rest/track-order/:order_id`

DESCRIPTION

The order-history functionality does not properly enforce object-level authorization. An authenticated user can manipulate order identifiers to retrieve order details belonging to other users.

STEPS TO REPRODUCE

1. Log in as a victim user and buy any item.

2. Note the order id (tracking id) when the order is placed.

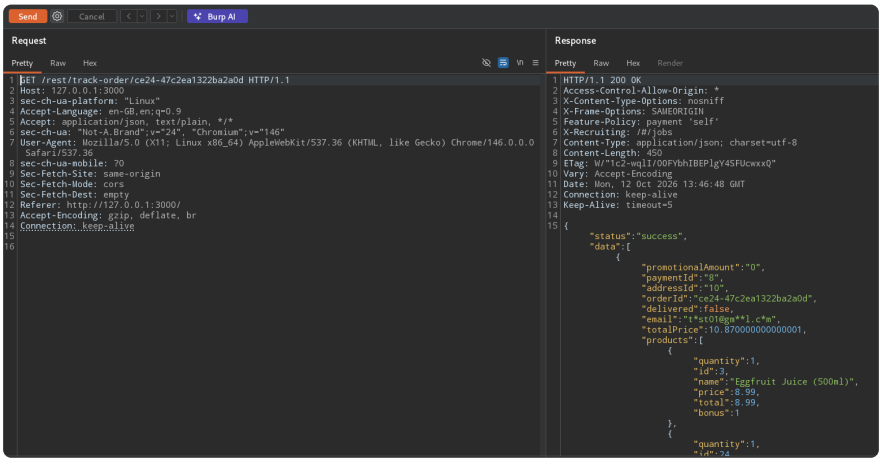
3. Navigate to **Orders & Payment → Order History** and capture the API request in burpsuite which is used to retrieve order details.

4. Modify the order identifier to reference an order owned by another user:

```
GET /rest/track-order/<victim_order_id>
```

5. Send the modified request. The application returns the other user's order information without verifying ownership or authentication.

PROOF



IMPACT

Unauthorized disclosure of sensitive customer and transactional data, including purchased products, quantities, prices, delivery details, and potentially payment-related metadata. This may violate privacy requirements and expose business-sensitive information.

REMEDIATION

SHORT/LONG TERM: Enforce server-side authorization on every order request. Validate that the authenticated user owns

the requested order before returning data, avoid relying on client-supplied object identifiers for authorization, and return consistent errors for unauthorized access attempts.

SEC-06: Forged User Feedback / Product Review

SEVERITY
High (7.5)

VULNERABILITY CLASS
Improper Authorization / Business Logic Abuse

CWE IDENTIFIER
CWE-282

AFFECTED ENDPOINT
`/rest/products/1/reviews`

DESCRIPTION

The feedback and product-review submission functionality does not adequately bind submitted records to the authenticated user or validate the integrity of user-controlled fields. An attacker can manipulate the request to submit feedback or reviews attributed to another user, bypass intended restrictions, or forge review content.

STEPS TO REPRODUCE

1. Log in as a normal user and submit a product review or feedback entry.

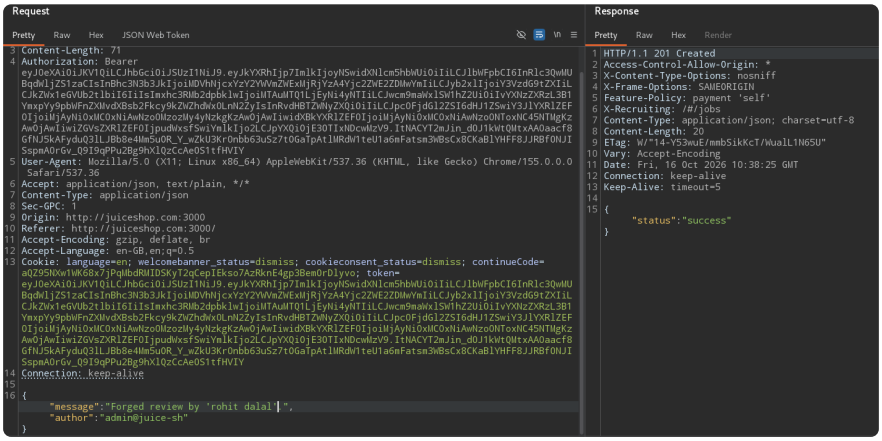
2. Intercept the submission request using Burp Suite.

3. Modify fields such as `message` , `author` For example:

```
{"message":"Forged review by 'rohit dalal'.","author":"admin@juice-sh"}
```


4. Forward the modified request. The application accepts and stores the forged feedback or review.

PROOF



IMPACT

Attackers can manipulate product reputation, impersonate other users, post misleading reviews, or abuse feedback mechanisms. This undermines trust in customer-generated content and can be used for fraud, harassment, or reputational damage.

REMEDIATION

SHORT TERM: Derive the author identity exclusively from the authenticated server-side session, never from client-supplied fields.

LONG TERM: Enforce server-side validation of permitted fields, implement authorization checks for create/update/delete operations, and apply rate limiting and abuse monitoring.

SEC-07: Cross-Site Request Forgery in Password Change

SEVERITY
High (8.1)

VULNERABILITY CLASS
Cross-Site Request Forgery (CSRF)

CWE IDENTIFIER
CWE-352

AFFECTED ENDPOINT
`/rest/user/change-password`

DESCRIPTION

The password-change operation can be triggered cross-site without a valid anti-CSRF token or sufficient same-origin protection. Because the request is not adequately bound to an attacker-resistant user interaction, a malicious website can cause an authenticated victim's browser to submit a password-change request.

STEPS TO REPRODUCE

1. Log in as a test user and capture a legitimate password-change request.

2. Observe that the state-changing request lacks a CSRF token and relies on ambient authentication credentials.

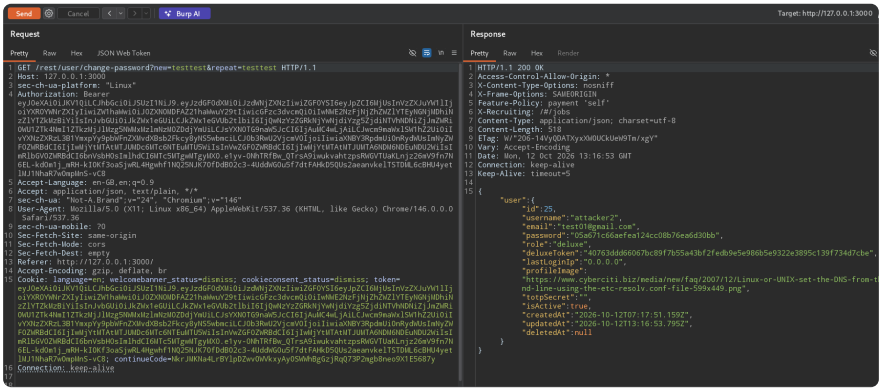
3. Host the following HTML page on an attacker-controlled origin and send it to the victim via email.

```
<html>
  <body>
    <form action="http://juiceshop:3000/rest/user/
change-password">
      <input type="hidden" name="new"
value="NewPass&#64;1" />
      <input type="hidden" name="repeat"
```

```
value="NewPass&#64;1" />
      <input type="submit" value="Submit request" />
    </form>
    <script>
      document.forms[0].submit();
    </script>
  </body>
</html>
```

4. The victim open recieved URL with their existing authentication context, changing the account password without their informed consent.

PROOF



IMPACT

An attacker can take over a victim's account by changing its password. This may result in loss of account access, unauthorized transactions, exposure of personal data, and further attacks against the victim.

REMEDIATION

LONG TERM: Sensitive endpoints such as password change must include `CSRF Token` in each requests.
Validate the `Origin` of request and `SameSite=Lax` or `Strict` cookies as defense in depth.

SEC-08: IDOR Allows Viewing Other Users' Shopping Baskets

SEVERITY
High (7.7)

VULNERABILITY CLASS
Insecure Direct Object Reference (IDOR)

CWE IDENTIFIER
CWE-639

AFFECTED ENDPOINT
`/rest/basket/<id>`

DESCRIPTION

The basket retrieval endpoint uses a predictable numeric basket identifier and fails to verify that the requested basket belongs to the authenticated user. By changing the identifier, an attacker can retrieve another user's basket contents.

STEPS TO REPRODUCE

1. Log in as a normal user and open the basket page.

2. Capture the basket API request:

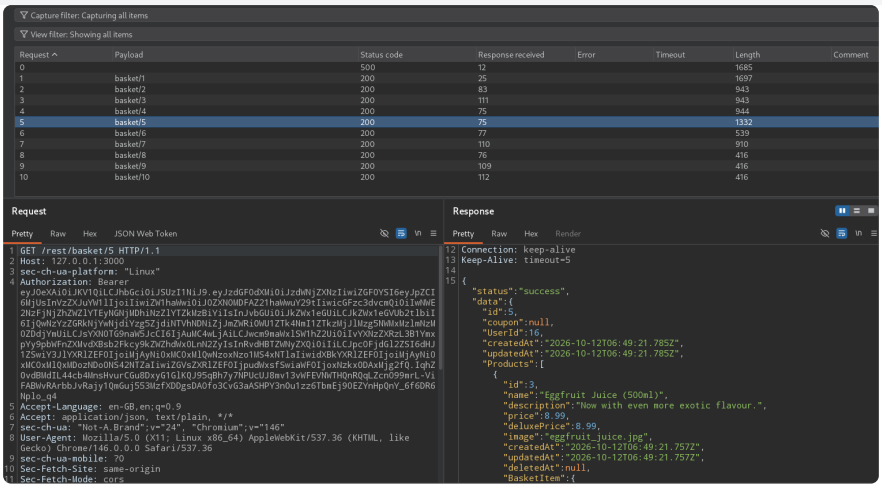
```
GET /rest/basket/6
```

3. Change the basket identifier to another user's basket ID:

```
GET /rest/basket/1
```

4. Send the request. The server returns the contents of the other user's shopping basket.

PROOF



IMPACT

Unauthorized disclosure of other users' basket data, including selected products, quantities, and pricing information. This may reveal user activity and can facilitate targeted fraud or social-engineering attacks.

REMEDIATION

LONG TERM: Implement object-level authorization checks confirming that the authenticated user owns the requested basket. Avoid exposing sequential internal identifiers where possible, and use unique UUIDs

SEC-09: Unauthorized Order Tracking of Other Users

SEVERITY

High (7.7)

VULNERABILITY CLASS

Broken Access Control / Information Disclosure

CWE IDENTIFIER

CWE-639

AFFECTED ENDPOINT

```
/rest/track-order/<id>
```

DESCRIPTION

The order-tracking endpoint does not verify that the requester is authorized to view the supplied order identifier. An unauthenticated or low-privileged user can enumerate order IDs and obtain tracking information for orders belonging to other customers.

STEPS TO REPRODUCE

1. Access the order-tracking page or directly call the tracking endpoint from the burpsuite.

2. Submit or request an order identifier belonging to another user:

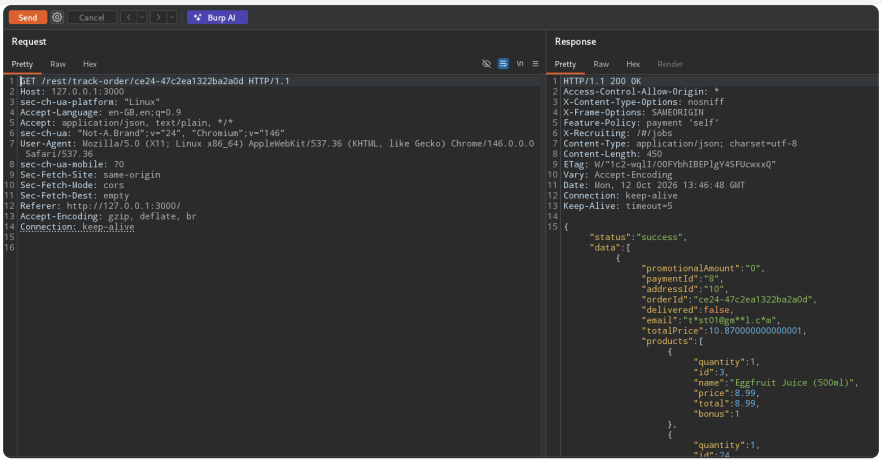
```
GET /rest/track-order/1
```

3. Increment or modify the order identifier to enumerate additional records:

```
GET /rest/track-order/2
GET /rest/track-order/3
```

4. The application returns order and delivery-tracking details without enforcing ownership or authentication.

PROOF



IMPACT

Exposure of customer order and shipment information to unauthorized parties. This may disclose purchase behavior, delivery addresses or timing information, and other sensitive transactional details.

REMEDIATION

SHORT TERM: Require authentication for order tracking and verify that the requested order belongs to the authenticated user. Use non-enumerable order references (`UUID`), enforce rate limiting.
LONG TERM: Log repeated unauthorized lookup attempts (SOC Team) and block any malicous activities.

SEC-10: FTP Dir Listing is Enabled And Contains Critical Files (`.kdbx`)

SEVERITY
Low (4.1) - Medium (6.5)

VULNERABILITY CLASS
Data Exposure / Misconfigured Directory Listing

CWE IDENTIFIER
CWE-538

AFFECTED ENDPOINT

`http://juiceshop.com:3000/ftp`

DESCRIPTION

The application exposes an FTP directory with directory listing enabled. Confidential files, including a KeePass database file with a `.kdbx` extension, are publicly discoverable and downloadable with other files. Exposure of a password database materially increases the risk of credential compromise, particularly if the database uses a weak or guessable master password.

STEPS TO REPRODUCE

1. Browse to the exposed FTP service:

```
http://juiceshop.com:3000/ftp
```

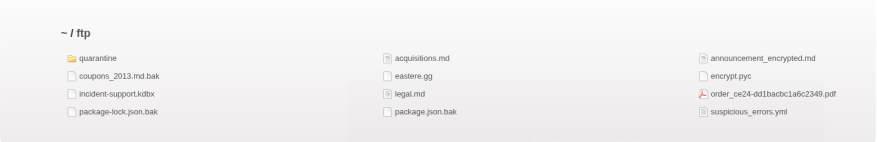
2. Observe that directory indexing is enabled and confidential files are listed.

3. Download the exposed KeePass database file:

```
http://juiceshop.com:3000/ftp/incident-support.kdbx
```

4. Confirm that the downloaded file is a KeePass database. If the master password is weak or known, extract stored credentials using an appropriate password-recovery or database-opening tool.

PROOF



IMPACT

Public exposure of a credential vault can lead to compromise of multiple application, infrastructure, or administrative accounts. Successful decryption of the KDBX file may provide attackers with persistent, privileged access across connected systems.

REMEDIATION

SHORT TERM: Disable `/ftp` directory listing, remove confidential files from publicly accessible storage.
LONG TERM: Store secrets in a managed secrets vault, rotate any potentially exposed credentials, and ensure KDBX databases use strong master password practice.

5. CONCLUSION

The penetration test successfully identified and documented critical vulnerabilities within the target environment. Applying the recommended mitigations will greatly enhance the security profile of the system. A follow-up verification test is recommended once patches are applied.

© 2026 Security Assessment.